The interesting thing here is that in theory, the workflow could fail for a variety of reasons (most likely in this case a validation failure or a networking error reaching the database), but no matter how the workflow fails, the controller responds in basically the same way: redirect and display an error message.

Because the controller's response is consistent—you only need to test at the controller level for one error response—you can test for different error cases at the workflow level. This is great because the workflow test is likely to be much faster than the integration test, so having multiple workflow tests is better for the system's runtime.

mocks/01/spec/workflows/creates_project_spec.rb

```
describe "mocking a failure" do
  it "fails when we say it fails" do
    project = instance_spy(Project, save: false)
    allow(Project).to receive(:new).and_return(project)
    creator = CreatesProject.new(name: "Name", task_string: "Task")
    creator.create
    expect(creator).not_to be_a_success
  end
end
```

Let's think about this test from a given/when/then perspective:

- Given: The test needs some way to simulate failure. In the earlier tutorial, you used a set of form values that were invalid. In this case you'll use a stub. The tricky part is that you need to make sure your stubbed project is the actual project instance that gets used in the workflow. Since the workflow creates its own instance, this is a little tricky, which is why you need to both create an `instance_spy` for the project and stub the `Project` class to return the spy when it receives a `new` message. Another way to do this would be to use `allow_any_instance_of(Project)`. You also might see the use of *dependency injection*, where the behavior of `CreatesProject` creating an instance of `Project` would be set as a variable at runtime—usually something like `def initialize(initialize(name: "", task_string: "", class_to_create: Project)`. Allowing the class to be specified dynamically allows the use of a class double or fake class in testing.

- When: The "when" action in this test is the `create` call on the workflow. When the ActiveRecord `save` happens inside the `CreatesProject` object, you need to simulate a save failure.

- Then: The temptation is to test that new objects are not created and existing objects are not updated. It makes no sense to test that the spy doesn't get saved. It can't get saved; it's not an ActiveRecord model. Instead, the test needs to make sure the workflow correctly returns its